Cristaux Jewellery

Assignment 2 — Short Report: HTTP APIs, gRPC & Metrics

Enterprise Software Development, Fall 2026

This report accompanies the code repository. Each section references the files where the deliverable lives.

1. REST Principles — The Six Constraints

Six REST constraints from the lecture. Code on Demand is optional. Each has a plain-language meaning and one example from this API.

#	Constraint	Plain-language meaning	Example in this API
1	Client–Server	UI and data storage live in separate components so each can evolve independently.	The browser storefront (GET /) is pure HTML/JS calling the REST API (/api/v1/...). The server owns the DB and business rules.
2	Stateless	Each request contains everything it needs; no client session is stored between requests.	Each order request is self-contained via the Idempotency-Key header. Not fully demonstrated: the single-instance rate limiter keeps a per-IP window in memory.
3	Cacheable	Responses must declare whether they may be cached.	Not strongly demonstrated. GET endpoints have safe semantics but no explicit Cache-Control headers are set.
4	Uniform Interface	A consistent way to address resources: standard methods, resource paths, consistent representations.	Uniform JSON error envelope {error:{code,message,details,timestamp}}; correct methods (GET/POST/PATCH/DELETE); Location header on 201 Created.
5	Layered System	A client cannot tell whether it is talking to the end server or an intermediary.	App runs behind Docker; storefront calls /api/v1/... regardless of proxy. The gRPC service is another layer sharing the same DB.
6	Code on Demand (optional)	The server may ship executable code to extend the client at runtime.	GET / returns inline JavaScript that runs in the browser and calls the API. Marked optional per the brief.

2. OpenAPI Specification — Explanation and Use

The OpenAPI Specification is a machine-readable contract (JSON/YAML) that describes an API's routes, request/response schemas, status codes, and error shapes. Tools use it to generate interactive documentation and client SDKs, and to drive automated testing. In this project FastAPI auto-generates the spec at /openapi.json, served through Swagger UI and ReDoc, and export_openapi.py writes it to disk.

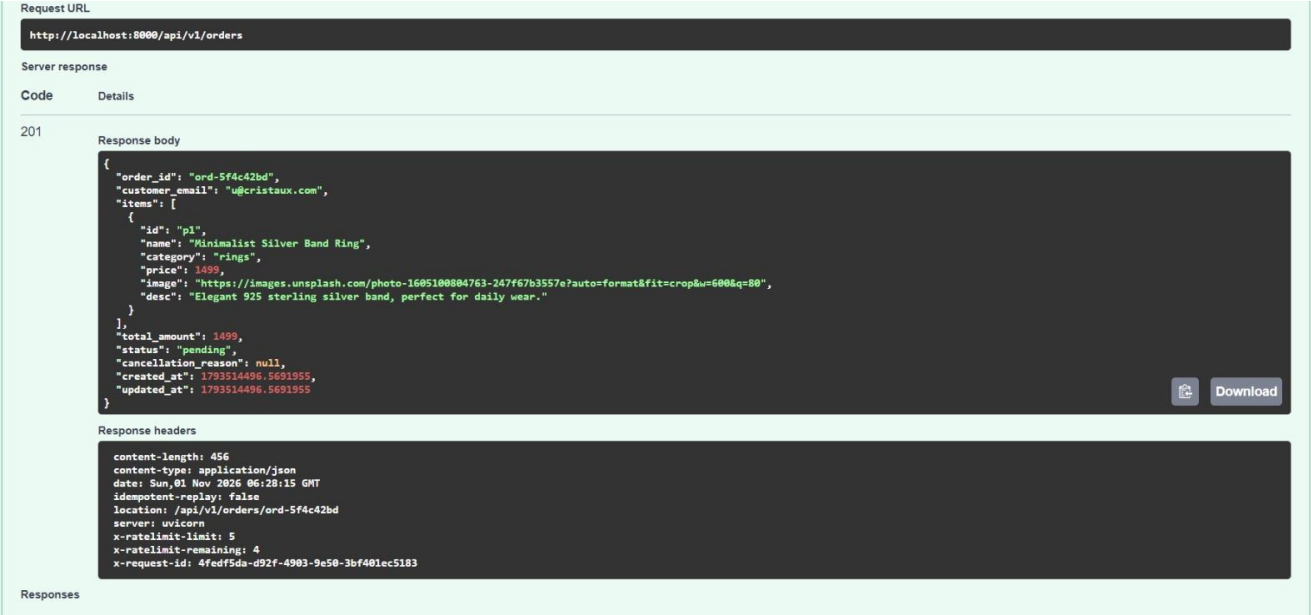
Using it to test an endpoint:

Open Swagger UI at http://localhost:8000/docs, expand POST /api/v1/orders, click "Try it out", paste a body {"cart_id":"sw-cart-1","customer_email":"u@cristaux.com"} and an Idempotency-Key header, then "Execute". Swagger sends the request and shows the 201 Created response plus the Location and Idempotent-Replay headers, confirming the live API matches the spec.

Spec location: openapi.json (repo root) + http://localhost:8000/openapi.json. Export command:

```
py -3 export_openapi.py
```

Evidence — Swagger UI executing the request live (201 Created + Location + Idempotent-Replay headers):



3. API Before / After — User Task: Place an Order

Old endpoint / behaviour	Problem	Change	Reason
POST /orders → 200 OK, no Location	Wrong success code; client cannot address the new order; no versioning.	POST /api/v1/orders → 201 Created + Location: /api/v1/orders/{id}	201 + Location is REST-correct for creation; versioning allows future evolution.
No idempotency — retry creates duplicate paid order	Timeouts / double-clicks double-charge customers.	Idempotency-Key header; replays return cached order with Idempotent-Replay: true.	POST is not safe; the key makes retries safe with no duplicate effects.
No rate limiting	Accidental floods or abuse exhaust the service.	5 orders / 10s per client IP; 6th returns 429 + Retry-After.	Protects the service from abuse and accidental floods.
Ad-hoc error bodies	Callers cannot reliably parse failures.	Uniform {error:{code,message,details,timestamp}} for all errors.	Predictable error contract for every caller.

Other endpoints changed or left alone

Changed: all application endpoints moved to versioned /api/v1/... paths with correct methods, Pydantic input validation (422 VALIDATION_ERROR) and the uniform error format. The RPC-style POST /orders/{id}/cancel and /fulfill became PATCH /api/v1/orders/{id} with a status body. Legacy aliases remain so old callers still work. **Left alone:** /health and /metrics (per the brief) and the A1 admin endpoints /admin/fault, /admin/cardinality-test and /admin/reset-limits. **Callers updated:** the storefront page, test_api.py and caller.py now call /api/v1/....

Method	Path	Purpose	Status codes
GET	/api/v1/products	List catalogue (?category= filter)	200
GET	/api/v1/products/{product_id}	Get one product	200, 404
GET	/api/v1/cart/{cart_id}	View cart	200, 404
POST	/api/v1/cart/items	Add item to cart	200, 404
DELETE	/api/v1/cart/{cart_id}	Clear cart	200
GET	/api/v1/orders	List orders	200
GET	/api/v1/orders/{order_id}	Get one order	200, 404
POST	/api/v1/orders	Place order (idempotent, rate-limited)	201, 400, 404, 422, 429
PATCH	/api/v1/orders/{order_id}	Cancel / fulfil order (replaces POST /orders/{id}/cancel and /fulfill)	200, 404, 409

4. Rate Limiting & Reliability Test Results

Rate limiting — Rate limiting on POST /api/v1/orders: 5 orders per 10-second rolling window per client IP (single-instance). The 6th request returns 429 Too Many Requests with Retry-After. Calls are allowed again after the oldest request ages out (≤10s).

```
Request #1: Status=201, X-RateLimit-Remaining=4
Request #2: Status=201, X-RateLimit-Remaining=3
Request #3: Status=201, X-RateLimit-Remaining=2
Request #4: Status=201, X-RateLimit-Remaining=1
Request #5: Status=201, X-RateLimit-Remaining=0
Request #6: Status=429, X-RateLimit-Remaining=0, Retry-After=10
>>> TEST 4 PASSED: accepted 5, rejected 6th with 429 & Retry-After.
```

Evidence — rate limiter returning 429 with Retry-After header:



Retry-After reflects the time remaining in the 10-second window when this request was made, so it differs from the value in the log above.

Automated tests: valid, invalid and repeated requests (py -3 test_api.py)

```
TEST 1 Valid:    add to cart 200; POST /api/v1/orders -> 201 + Location, Idempotent-Replay: false
TEST 2 Invalid:  empty cart -> 400 BAD_REQUEST; bad email -> 422 VALIDATION_ERROR;
                 negative quantity -> 422 VALIDATION_ERROR (all in the uniform error envelope)
TEST 3 Repeated: 1st call 201, Order ID ord-3793fe8c, Replay: false
                 2nd call (same Idempotency-Key) 201, same Order ID, Replay: true
                 database holds exactly 1 order for that ID (no duplicate)
TEST 4 Rate limit: 5 accepted, 6th -> 429 + Retry-After   (output shown above)
>>> ALL 4 TESTS PASSED
```

Idempotency: repeating POST /api/v1/orders with the same Idempotency-Key returns the cached order (201, Idempotent-Replay: true) instead of creating a second paid order, which is what makes the caller's retries safe.

Resilient Python caller (caller.py) — Settings: timeout 1.5s; retry limit 3; backoff exponential 0.5s→1.0s→2.0s; circuit breaker threshold 3 failures; recovery cooldown 4.0s. Idempotency-Key makes retries safe. Test uses injected HTTP 503 fault.

```
PHASE 1 (CLOSED):      Call #1, #2  →  201 Created (succeeding)
PHASE 2 (503 injected): Call #3,4,5  →  3 retries each, all fail
```

```
                        [CircuitBreaker] 3/3 failures → OPEN
PHASE 3 (OPEN):        Call #6, #7  →  FAST FAIL, blocked locally
PHASE 4 (recover):     wait 4s → canary → OPEN→HALF-OPEN→201→CLOSED
                        Call #9  →  201 (fully recovered)
>>> ALL PHASES VERIFIED: succeed / retry / block / recover
```

5. Call-Path Diagrams (HTTP and gRPC)

Both call paths include the generated stub and the server handler. Both converge on app/orders_repository.py + SQLite, sharing business rules and a single source of truth.

HTTP REST path (FastAPI):

```
Python client (requests.Session)
   |  PATCH /api/v1/orders/{order_id}   header: x-request-id
   v
FastAPI middleware  (app/main.py: observability_middleware)
   - generates / propagates x-request-id
   - records http_request_duration_seconds histogram (server-side)
   - emits structured JSON log: {request_id, status_code, duration_ms}
   v
update_order_status_v1()  ->  db_cancel_order(order_id, reason)
   v
app/orders_repository.py   (shared business rules)
   - returns NOT_FOUND | ALREADY_CANCELLED | ALREADY_FULFILLED | SUCCESS
   v
SQLite  (data/cristaux_orders.db, rollback journal)   <-- shared source of truth
   v
HTTP 200 OK | 404 Not Found | 409 Conflict
   v
Filebeat -> Elasticsearch / Kibana  (container.name = "app")
```

gRPC path (Part B standalone process):

```
grpc_client.py   (argparse CLI, deadline, x-request-id metadata)
   |
   |  order_service_pb2_grpc.OrderServiceStub(channel)   <-- GENERATED STUB
   |  stub.CancelOrder(request, timeout=deadline,
   |                   metadata=(("x-request-id", req_id),))
   v
HTTP/2 + protobuf   ->   127.0.0.1:50051
   v
grpc.server(ThreadPoolExecutor)   in grpc_server.py
   v
OrderServiceServicer.CancelOrder(request, context)   <-- SERVER HANDLER
   - extract_request_id(context)  reads invocation metadata
   - structured JSON log: {request_id, rpc, status_code, duration_ms}
   - db_cancel_order(order_id, reason)   SAME shared business rules
   v
app/orders_repository.py   (shared business rules)   SAME module as REST
   v
SQLite  (data/cristaux_orders.db, rollback journal)   <-- SAME source of truth
   v
CancelOrderResponse  OR  context.abort(NOT_FOUND / FAILED_PRECONDITION)
   + context.set_trailing_metadata(("x-request-id", request_id))
   v
gRPC status: OK(0) | NOT_FOUND(5) | FAILED_PRECONDITION(9) | DEADLINE_EXCEEDED(4)
   v
Filebeat -> Elasticsearch / Kibana  (container.name = "grpc-service")
```

SQLite runs in the default rollback-journal mode (journal_mode=DELETE) rather than WAL: WAL needs a shared-memory file that is unreliable when the database is bind-mounted into Docker containers on Windows, and the REST and gRPC containers share the same database file.

Code generation command (reproduces the committed stubs):

```
py -3 -m grpc_tools.protoc -I. --python_out=. --grpc_python_out=. proto/order_service.proto
```

Contract (proto/order_service.proto, proto3, package cristaux.orders):

```
service OrderService {
  rpc CancelOrder (CancelOrderRequest) returns (CancelOrderResponse);
  rpc GetOrder    (GetOrderRequest)    returns (GetOrderResponse);
}
message CancelOrderRequest  { string order_id = 1; string reason = 2; }
message CancelOrderResponse { string order_id = 1; string status = 2; string reason = 3;
                              string updated_at = 4; string message = 5; }
message GetOrderRequest     { string order_id = 1; }
message OrderItem           { string id = 1; string name = 2; string category = 3;
                              double price = 4; }
message GetOrderResponse    { string order_id = 1; string customer_email = 2;
                              double total_amount = 3; string status = 4;
                              string cancellation_reason = 5; string created_at = 6;
                              repeated OrderItem items = 7; }
```

Evidence — structured JSON log lines emitted by the grpc-service container, carrying request_id, rpc, status_code and duration_ms:


```
{"@timestamp": "2026-11-01T07:50:57.971Z", "service": "Cristaux Jewellery gRPC Service", "level": "INFO", "message": "gRPC CancelOrder succeeded", "request_id": "seed-20-5fa8f405", "rpc": "CancelOrder", "order_id": "ord-seed-c585d9", "status_code": "OK", "duration_ms": 91.56}
{"@timestamp": "2026-11-01T07:50:57.982Z", "service": "Cristaux Jewellery gRPC Service", "level": "WARNING", "message": "gRPC CancelOrder failed: order not found", "request_id": "seed-20-5fa8f405", "rpc": "CancelOrder", "order_id": "ord-ghost-99999999", "status_code": "NOT_FOUND", "duration_ms": 7.76}
{"@timestamp": "2026-11-01T07:50:57.991Z", "service": "Cristaux Jewellery gRPC Service", "level": "WARNING", "message": "gRPC CancelOrder failed: order already cancelled", "request_id": "seed-20-5fa8f405", "rpc": "CancelOrder", "order_id": "ord-seed-c585d9", "status_code": "FAILED_PRECONDITION", "duration_ms": 5.73}
```


Output of docker logs grpc-service (3 of the last 6 lines shown): one successful call, one NOT_FOUND and one FAILED_PRECONDITION, each with request_id, rpc, status_code and duration_ms.

Trace correlation across the gRPC boundary

The client generates a request_id and sends it as call metadata (x-request-id). The server reads it in extract_request_id(context) and writes it into every structured JSON log line (request_id, rpc, order_id, status_code, duration_ms), using the same schema as the FastAPI middleware. On successful calls the server also returns it in the trailing metadata so the client can match its own records. Filebeat lifts request_id to a top-level field for both the app and grpc-service containers, so one request_id filter in Kibana shows both legs of an operation.

6. HTTP and gRPC Request Examples

Run via: py -3 test_grpc_vs_http.py

1) Successful cancellation of a pending order

```
[HTTP REST]  PATCH /api/v1/orders/ord-http-succ-73c726
  HTTP Status: 200 OK
  Response: {'order_id':'ord-http-succ-73c726','status':'cancelled'}

[gRPC]  cristaux.orders.OrderService/CancelOrder
  gRPC Status: OK (0)
  Response: order_id='ord-grpc-succ-bb67e2', status='cancelled'
```

```
>>> Both orders verified as 'cancelled' in shared SQLite database.
```

2) Failed — non-existent order

```
[HTTP REST]  Status 404 Not Found    error.code = RESOURCE_NOT_FOUND
[gRPC]        Status NOT_FOUND (5)    "Order 'ord-ghost-99999999' was not found in the database."
```

3) Failed — invalid state (already cancelled)

```
[HTTP REST]  Status 409 Conflict     error.code = CONFLICT
[gRPC]        Status FAILED_PRECONDITION (9)    "Order '...' is already in cancelled state."
```

4) Failed — client deadline exceeded (gRPC)

```
py -3 grpc_client.py --deadline 0.001 --host 127.0.0.1:50051
  gRPC Status: DEADLINE_EXCEEDED (4)   (HTTP equivalent: 408 / 504)
```

Status code mapping

Scenario	HTTP Status	gRPC Status Code	Meaning
Successful cancel	200 OK	OK (0)	Operation fulfilled; order state modified.
Missing order	404 Not Found	NOT_FOUND (5)	order_id does not exist in source of truth.
Already cancelled	409 Conflict	FAILED_PRECONDITION (9)	Order not in valid pending state.
Client timeout	408 / 504	DEADLINE_EXCEEDED (4)	Server did not reply within deadline.
Server unreachable	connection refused	UNAVAILABLE (14)	gRPC server not reachable at host:port.

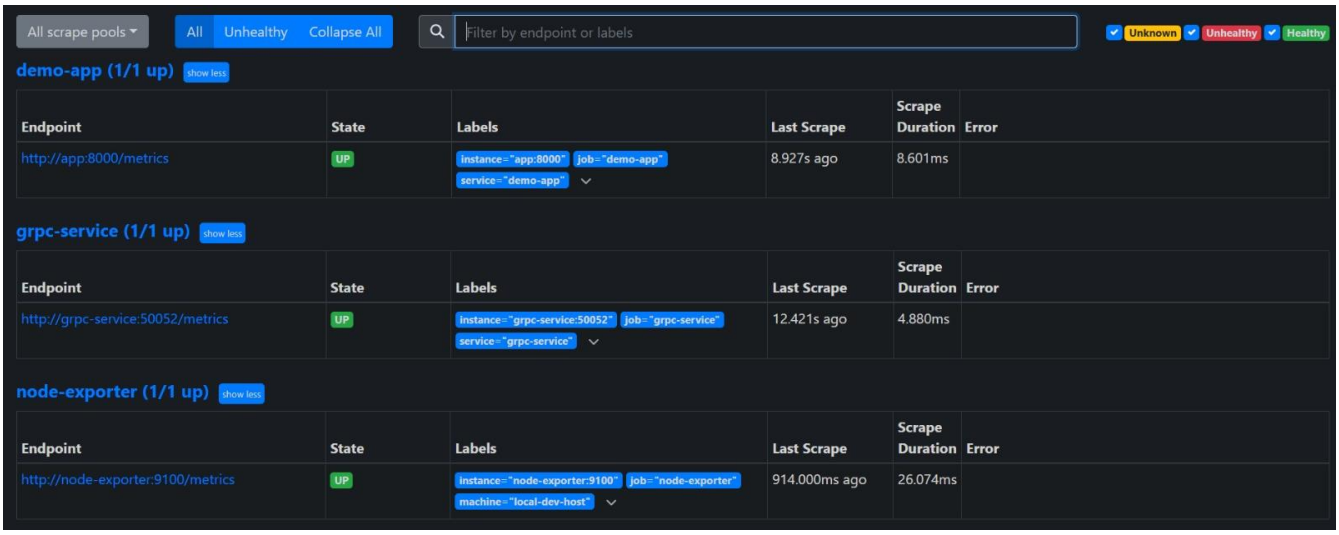
7. Metric Gap Table — A1 Audit

Audit of Assignment 1 metrics: what was useful, what was missing, and what was fixed or added in Part C.

Useful, kept as-is: jewellery_orders_placed_total{category} (counter), jewellery_orders_cancelled_total{reason} (counter), jewellery_pending_orders_count (gauge, fulfilment queue depth), jewellery_active_carts_count (gauge), http_request_duration_seconds{method,path,status} (histogram, HTTP latency) and jewellery_order_processing_seconds (summary, checkout time). The gaps and fixes are below.

Issue / Metric	Finding	Fix / addition
http_request_duration_seconds used raw request.url.path	Embedded real order_id / cart_id → unbounded label cardinality.	normalize_path() uses route template (/api/v1/orders/{order_id}); path label set is bounded.
gRPC service had no metrics	Prometheus could not scrape Part B; no calls / duration / concurrency visibility.	Added /metrics on port 50052 + 3 new gRPC metrics; added grpc-service scrape job.
No error-rate by method/status	Error rate not derivable from A1 metrics.	grpc_calls_total{rpc_method,status}; error rate = rate(...{status!="OK"}).
No concurrency metric	No visibility into in-flight work.	Added grpc_in_flight_calls{rpc_method}. Chosen over message size (explained in Section 8).
Only pending count tracked	No order-state distribution (cancelled/fulfilled missing).	Added jewellery_orders_by_status{status} computed from shared SQLite DB at scrape time.
jewellery_orders_cancelled_total{reason}	reason is semi-unbounded (customer-provided string).	Flagged; kept for A1 compatibility. Would normalize to enum if it grew.
demo_requests_total{request_id} (A1 Part E)	Intentionally unbounded — the cardinality-explosion example.	Not fixed; it is the demonstration of the anti-pattern.

Evidence — Prometheus targets page confirming both app (port 8000) and grpc-service (port 50052) are scraped successfully:



8. New Metrics and PromQL Queries

Duration is measured at the SERVER for both transports. In-flight chosen over message size: unary RPCs have small, fixed-size protobuf payloads so message size is near-constant; in-flight reveals concurrency and stuck handlers. All labels are bounded — request_id / trace IDs are never used as labels.

Histogram buckets: 5 ms to 2.5 s (0.005, 0.01, 0.025, 0.05, 0.1, 0.25, 0.5, 1.0, 2.5 s). These suit unary RPCs backed by local SQLite, which finish in about 5–90 ms in the logs above, so p50/p95 resolve well; the coarser upper buckets expose stalls. Duration is measured in the handler with perf_counter, from entry to return or abort. **Note:** DEADLINE_EXCEEDED is raised on the client, so it never appears in grpc_calls_total (the server only counts calls whose handler ran).

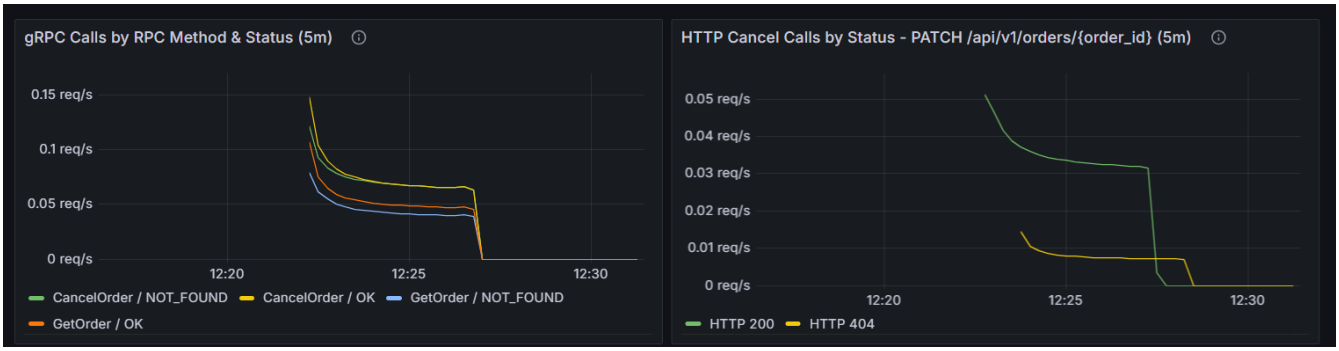
Metric	Purpose	Type/ Unit	Labels (bounded)	Code location	Client/Server	Example PromQL query
grpc_calls_total	Calls and error rate by RPC method and status.	Counter / calls	rpc_method ∈ {CancelOrder, GetOrder} status ∈ {OK, NOT_FOUND, FAILED_PRECONDITION, …}	grpc_server.py (record_rpc)	—	sum by (rpc_method, status) (rate(grpc_calls_total[5m]))
grpc_request_duration_seconds	Handler latency for p50 / p95.	Histogram / seconds	rpc_method, status	grpc_server.py (record_rpc)	Server	histogram_quantile (0.95, sum by (le, rpc_method) (rate(..._bucket[5m]))))
grpc_in_flight_calls	Concurrency; reveals stuck or slow handlers.	Gauge / calls	rpc_method	grpc_server.py (handler inc/dec in finally)	—	grpc_in_flight_calls
jewellery_orders_by_status	Order-state distribution (missing A1 metric).	Gauge / orders	status ∈ {pending, cancelled, fulfilled}	app/main.py (OrderStatusCollector) app/orders_repository.count_orders_by_status	—	jewellery_orders_by_status

9. Dashboard Evidence

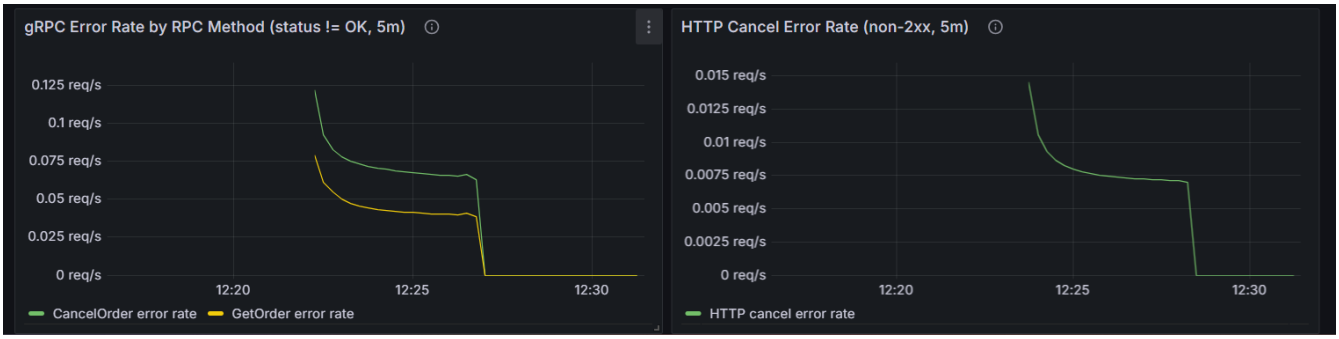
Dashboard: "Cristaux - Part C: gRPC vs HTTP Operation Metrics" at http://localhost:3000. Traffic generated via: py -3 seed_traffic.py and py -3 grpc_seed_traffic.py

All panels use a 5-minute time window. Duration is server-side for both transports.

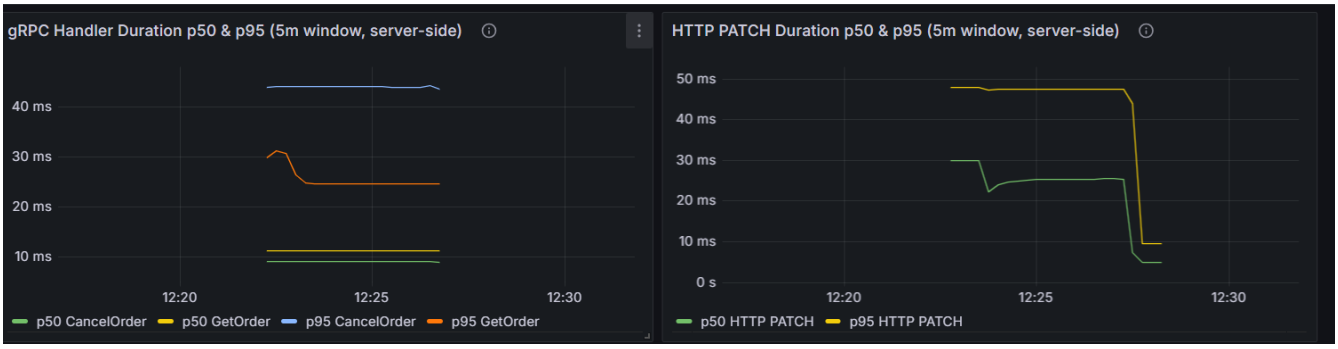
gRPC Calls by RPC Method & Status + HTTP Cancel Calls by Status (5m)



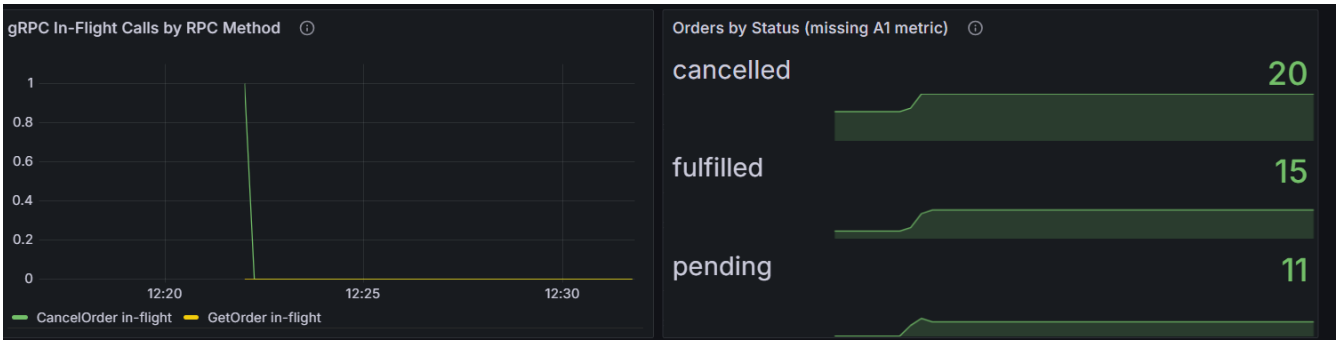
gRPC Error Rate + HTTP Cancel Error Rate (5m)



gRPC Handler Duration p50 & p95 + HTTP PATCH Duration p50 & p95 (5m, server-side)

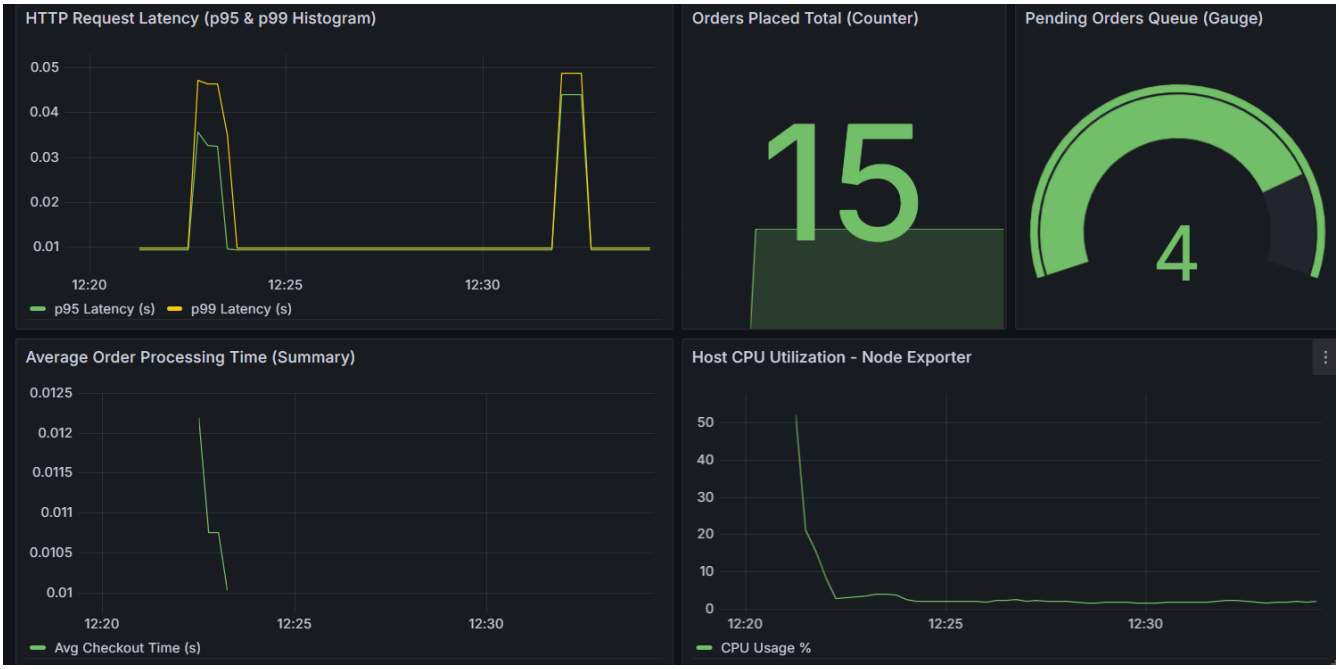


gRPC In-Flight Calls + Orders by Status (missing A1 metric)



In-flight calls are short-lived (handlers finish in milliseconds), so the gauge shows only a brief spike. Orders by Status reads the whole shared database, including orders created by the seed scripts.

A1 Dashboard — Jewellery Store Observability Dashboard



Note: all dashboard panels in this section were captured from the same traffic run (approx. 12:20–12:35). The Pending Orders Queue gauge on the A1 dashboard and the Orders by Status panel are different metrics (the latter is computed from the shared database at scrape time, see Section 7), so their counts differ. Rate-based panels use a 5-minute window and return to zero once traffic stops.

Reused work / academic integrity: The gRPC server/client structure follows the course gRPC workshop (week 1 Midnight Launch). No third-party code was copied; only standard grpcio / grpcio-tools patterns were reused. The data/ SQLite volume and Docker volumes are excluded from the submission (.gitignore).